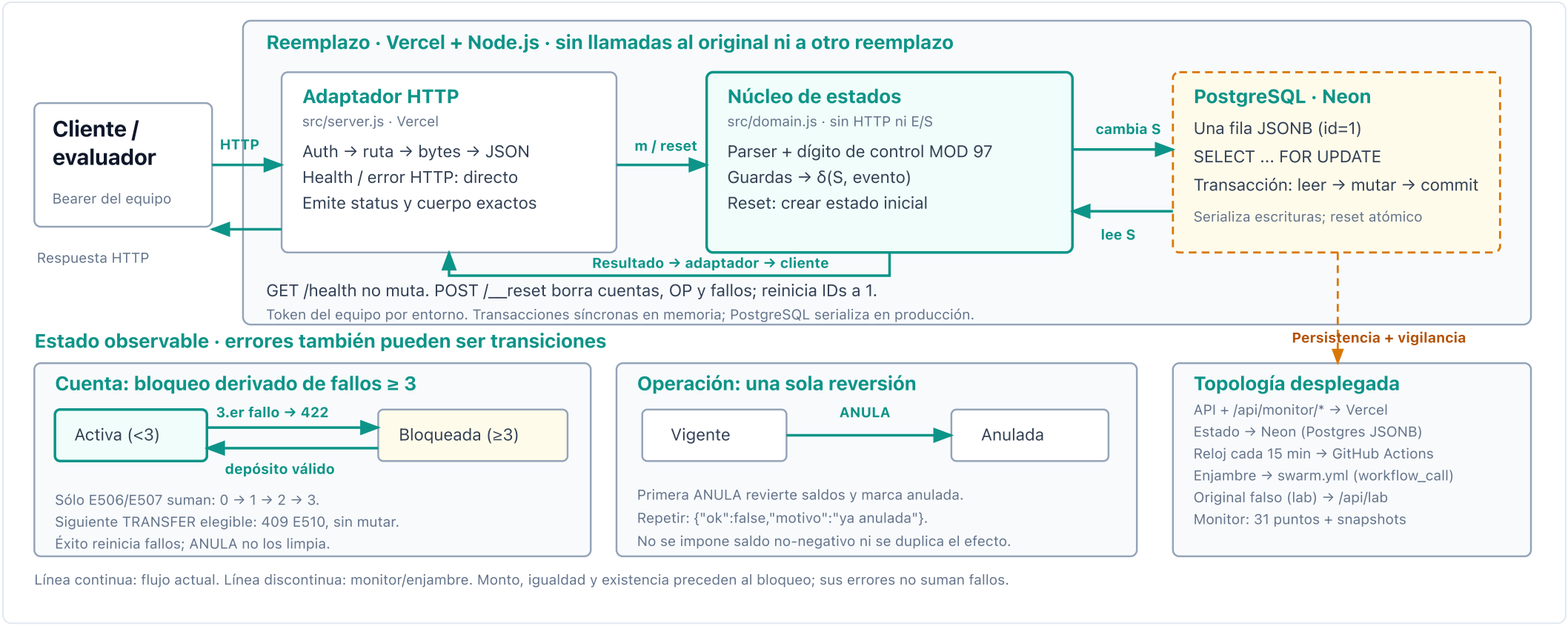
Arquitectura del reemplazo · Un proceso, estado explícito

Frontier Labs · Reto 2 · Systems Design (30%) · cierre 12:00 · runtime (Vercel + Neon) + vigilancia y enjambre (GitHub Actions)



Decisiones, alternativas y costes

Elección	Alternativa descartada / razón	Coste aceptado
Monolito, una instancia	Microservicios: coordinación sin necesidad del contrato.	Sin alta disponibilidad ni réplicas.
Dominio separado de HTTP	Reglas en rutas: dificulta pruebas y cambios de transporte.	Mantener la frontera de dos módulos.
Map local / PostgreSQL prod	DB única: infraestructura extra en tests sin beneficio. SQLite: driver/runtime por validar.	Dos adaptadores de persistencia; probar serialización en Neon.
JSONB de una fila (Neon)	Multi-tabla inicial: complejidad sin réplicas requeridas. SQLite con volumen: no cubre serverless nativo.	Una fila serializa escrituras; locks <5 s.
Conservar anomalías	Normalizar errores o invariantes nuevas: rompe compatibilidad.	Conservar typos y comportamientos observados.

Verificación por la superficie pública

Casos: entradas y precedencia; secuencias de 2–5 y 6–12 pasos; tercer fallo y bloqueo; depósito desbloqueando; primera/segunda ANULA; reset completo.

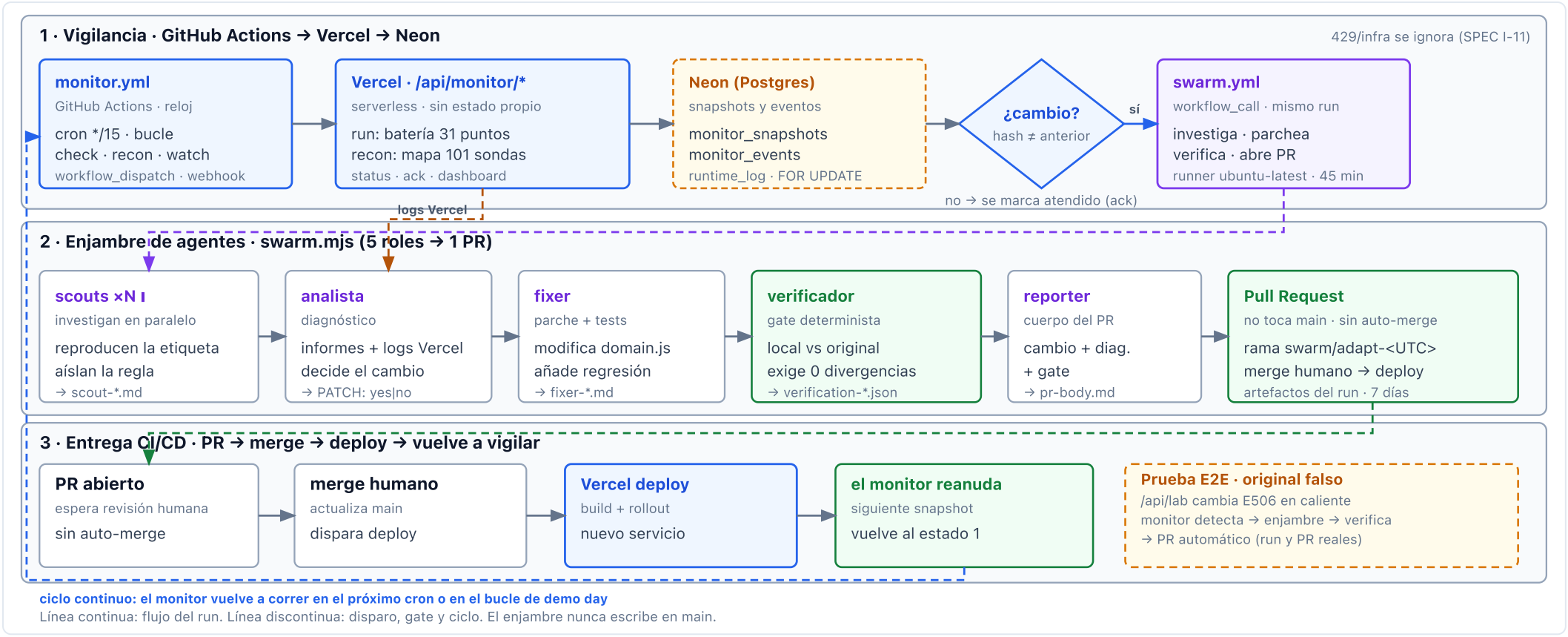
Aceptación: 100% de status/cuerpos y efectos posteriores coincidentes, cero timeouts y **cada request <5 s** en el endpoint público. Reset explícito por secuencia; no reintentar mutaciones a ciegas.

Adaptabilidad: transporte cambia en el adaptador; reglas en el núcleo; persistencia se adapta por `storage.js` sin rediseñar reglas.

Dígito de control MOD 97; no firma criptográfica. Fuentes: contrato oficial + `reto2-investigacion/SPEC.md` (366 observaciones) y `experimentos.jsonl`.

Vigilancia y adaptación autónoma · CI/CD + enjambre de agentes

Además de responder igual, el reemplazo **detecta cuándo el original cambia y propone su propio parche**. Nada de esto corre dentro del servicio: vive en GitHub Actions, exige **cero divergencias** y abre un **PR** que un humano revisa.



Línea continua: flujo del run. Línea discontinua: disparo, gate y ciclo. El enjambre nunca escribe en main.

CI/CD y enjambre · decisiones

Elección	Alternativa descartada / razón	Coste aceptado
GitHub Actions como runner	Vercel Functions: efímeras, sin git ni filesystem; no pueden editar, testear y abrir PR.	Máquina efímera por evento; job de 45 min.
`workflow_call` en el mismo run	`repository_dispatch` desde un workflow: `GITHUB_TOKEN` no encadena workflows.	Monitor y enjambre comparten run.
Verificación determinista como gate	Confiar en el LLM del fixer: puede alucinar.	Si no hay 0 divergencias, no hay PR.
El enjambre abre PR, no mergea	Auto-merge: un parche erróneo llegaría a producción.	Revisión humana; merge dispara deploy.

Cómo se prueba

- **Original falso** `/api/lab` cambia un comportamiento en caliente (p. ej. el límite de `E506`).
- El monitor lo detecta → `workflow_call` → el enjambre parchea, verifica y abre **PR real**.
- Verificación **no concluyente** (429/infra): no parchea ni falla el workflow; el evento queda pendiente.
- Artefactos por run: `changes.json`, `scout-*.md`, `diagnosis.md`, `fixer-*.md`, `verification-*.json`, `pr-body.md` (7 días).
- Límite asumido: un cambio en el **algoritmo de control** exigiría re-descifrarlo; el verificador lo detecta y el evento queda pendiente.

Detalle de roles, harness y configuración: `agent/README.md`. Disparadores y tablas: `docs/observabilidad.md`. Workflows: `.github/workflows/monitor.yml` y `swarm.yml`.

Complementa el documento de diseño. Implementación: `reemplazo/src/server.js`, `domain.js`, `storage.js`. Monitor/enjambre: `agent/README.md` y `docs/observabilidad.md`.